

硬件 WCCA 文档生成自动化规划方案

1. 方案背景

WCCA，即 Worst Case Circuit Analysis，通常用于分析硬件电路在最坏条件下是否仍能满足设计需求。其分析对象通常包括电源轨、分压网络、ADC 采样链路、比较器阈值、运放电路、RC 延时、电平接口、热设计、保护电路等。

传统 WCCA 文档生成依赖硬件工程师手工整理资料、查阅 datasheet、提取参数、建立计算表、撰写分析过程、生成报告和反复修改。该过程存在以下问题：

1. 资料来源分散，datasheet、BOM、原理图、公司规范、历史报告难以统一管理。
2. 参数提取和计算过程大量依赖人工，效率低且容易遗漏。
3. 计算表、报告正文、引用来源之间容易出现不一致。
4. 历史项目经验难以复用，新项目经常重复劳动。
5. WCCA 报告需要满足可追溯、可复现、可审计要求，仅靠大模型直接生成文档存在风险。
6. 设计评审和签核过程缺少标准化工具支撑。

因此，建议建设一套面向硬件工程团队的 WCCA 文档生成自动化方案。该方案应以工程计算和数据可追溯为核心，以 AI 作为编排和文档生成助手，以知识库作为依据支撑，以人工评审作为最终责任闭环。

2. 建设目标

本方案的总体目标是构建一套 **可复现、可追溯、可审计、可复用** 的硬件 WCCA 文档生成自动化系统。

具体目标包括：

- 1. 提升文档生成效率**
自动完成资料整理、参数抽取、计算表生成、报告草稿生成和一致性检查，减少硬件工程师重复性文档工作。
- 2. 保证计算结果可复现**
所有 WCCA 关键数值均由 Python、Excel、SPICE 或其他确定性计算工具生成，避免由 AI 直接编造或口算关键工程数据。
- 3. 保证参数来源可追溯**
每个关键参数应能够追溯到 datasheet、公司规范、设计需求、历史项目或工程师确认记录。
- 4. 保证报告结论可审计**
报告中的 Pass/Fail、Margin、风险项和最终结论应能够回溯到计算表、仿真结果和依据文档。
- 5. 沉淀企业知识资产**
将 WCCA 模板、计算模型、参数规则、历史案例和评审经验固化为团队可复用资产。
- 6. 支持后续平台化扩展**
前期以 CLI、模板和知识库为核心，后续可扩展到 Web 平台、审批流、PLM/ERP/EDA 集成和多 Agent 协作。

3. 适用范围

本方案适用于硬件研发团队在以下场景中进行 WCCA 文档生成和自动化分析：

1. 产品研发阶段的 WCCA 报告生成。
2. 电源、模拟、数字接口、采样链路、控制电路等常见硬件模块分析。
3. EVT、DVT、PVT、量产导入前的设计裕量评估。
4. 高可靠性产品的设计审查、质量审计和客户交付资料准备。
5. 历史 WCCA 报告复用和新项目模板化生成。
6. 硬件规范、器件参数、降额规则和设计经验的知识库建设。

本方案不建议在初期直接用于完全自动化替代工程师判断，尤其不建议用于未经审核的安全关键、生命关键或法规约束系统的最终工程结论。

4. 核心原则

WCCA 文档生成自动化必须遵循以下原则：

4.1 AI 不得编造关键工程参数

AI 可以辅助检索、提取、归纳和生成文字，但不得在没有明确依据的情况下自行生成关键器件参数、边界条件、降额系数或设计结论。

4.2 不得使用 Typical 值替代 Worst-Case 值

WCCA 分析应优先使用 datasheet 中的 Min、Max、Tolerance、Temperature Coefficient、Aging、Drift、Derating 等最坏情况参数。若只能获得 Typical 值，必须标记为风险项或待工程师确认项。

4.3 所有关键数值必须来自计算或仿真结果

报告正文中的 Nominal、Worst Min、Worst Max、Margin、Pass/Fail 等数值必须来自计算表、仿真结果或经过确认的结构化数据，不应由大模型自由生成。

4.4 所有结论必须可追溯

每个关键结论应能追溯到以下至少一种依据：

- 设计需求规格书
- Datasheet
- 公司规范
- 计算表
- 仿真结果
- 历史 WCCA 案例
- 工程师确认记录

4.5 工程师必须最终签核

AI 和自动化工具只负责辅助分析、生成草稿、校验一致性和提示风险。最终工程判断、风险接受和正式发布必须由硬件工程师、系统工程师或质量/合规工程师审核签核。

4.6 先保证可复现，再追求智能化

系统建设顺序应优先保证计算、校验、报告生成的可复现性，再逐步引入 AI 智能编排、自动参数抽取、多 Agent 协作和平台化能力。

5. 总体推荐架构

推荐采用以下总体架构：

知识库/RAG + WCCA Core CLI + Python/SPICE 计算引擎 + AI Agent 编排 + 报告模板 + 工程师签核

各部分职责如下：

模块	核心职责
知识库/RAG	管理 datasheet、公司规范、历史 WCCA、参数来源和引用依据
WCCA Core CLI	执行资料导入、参数结构化、计算、仿真调用、校验、报告生成和归档打包
Python/Excel/SPICE	负责确定性工程计算、表格生成、参数扫描和仿真分析
AI Agent	负责任务规划、资料理解、参数提取辅助、报告撰写、风险总结和审查辅助
Skill	固化 WCCA 流程规范、提示词、模板规则和检查清单
Subagent	在复杂项目中承担 Planner、Extractor、Reviewer、Writer 等专项角色
工程师	确认边界条件、审核参数来源、判断风险、签核最终结论

核心思想是：

CLI 负责工程确定性，AI 负责智能编排和表达，知识库负责依据支撑，工程师负责最终判断。

6. 工具形态选择

6.1 不建议只做成 Skill

Skill 适合固化流程、模板、检查清单和提示规则，但不适合作为 WCCA 自动化的唯一实现方式。

Skill 适合负责：

- WCCA 文档章节规范
- 参数提取规则
- 报告写作风格
- 检查清单

- 风险项描述模板
- Pass/Fail 说明规则
- 引用格式规范

Skill 不适合负责：

- Worst Case 数值计算
- RSS 计算
- Monte Carlo 计算
- 大规模 BOM/网表解析
- SPICE 仿真
- 报告数值一致性校验
- 审计日志和版本管理

结论：**Skill 必须有，但不能作为核心执行引擎。**

6.2 推荐以 CLI 作为核心

WCCA Core CLI 应作为系统核心执行层。原因是 WCCA 的关键要求是可复现、可测试、可审计，而 CLI 天然适合工程化集成。

CLI 适合负责：

- 项目初始化
- BOM 导入
- 网表解析
- 需求文件解析
- 参数结构化
- 单位归一化
- Worst Case 计算
- RSS 计算
- Monte Carlo 调用
- SPICE 仿真调用
- 计算结果生成
- 报告模板填充
- 一致性检查
- 归档包生成
- 日志记录

推荐 CLI 命令设计如下：

```
wcca init
wcca ingest
wcca extract
wcca normalize
wcca calculate
wcca simulate
wcca validate
```

```
wcca report
wcca package
```

典型调用流程如下：

```
wcca init --project motor-controller-wcca

wcca ingest
--bom ./input/bom.xlsx
--netlist ./input/design.net
--requirements ./input/requirements.docx
--template ./templates/company_wcca.docx

wcca extract --output ./work/wcca_input.yaml

wcca calculate
--input ./work/wcca_input.yaml
--output ./output/wcca_results.xlsx

wcca validate
--input ./work/wcca_input.yaml
--results ./output/wcca_results.xlsx

wcca report
--input ./work/wcca_input.yaml
--results ./output/wcca_results.xlsx
--output ./output/wcca_report.docx

wcca package --output ./release/wcca_package.zip
```

结论：CLI 应作为 WCCA 自动化的工程核心。

6.3 Subagent 适合在中后期引入

Subagent 适合复杂项目中的任务拆解和专项审查，但不应替代 CLI 进行关键计算。

推荐设置以下 Subagent：

Subagent	职责
WCCA Planner Agent	识别分析范围、电路模块、缺失资料和工作计划
Parameter Extraction Agent	辅助提取器件参数、标注来源、区分 Min/Max/Typical
Circuit Analysis Agent	匹配电路类型和计算模型
Calculation Review Agent	检查单位、边界条件、输入合理性和低裕量项
Report Writer Agent	根据模板生成 WCCA 报告草稿

结论：**Subagent 适合编排和审查，不适合作为计算正确性的唯一保障。**

7. 系统核心模块设计

7.1 输入资料管理模块

输入资料包括：

- 原理图
- 网表
- BOM
- Datasheet
- 设计需求规格书
- 公司 WCCA 模板
- 降额规范
- 可靠性规范
- 历史 WCCA 报告
- 测试条件
- 仿真结果
- 工程师补充说明

该模块应完成以下工作：

1. 统一项目资料目录结构。
2. 检查必要资料是否齐全。
3. 识别文件版本。
4. 建立输入资料索引。
5. 标记缺失资料和待确认资料。

推荐项目目录结构：

```
project-wcca/  
├── input/  
│   ├── schematic/  
│   ├── netlist/  
│   ├── bom/  
│   ├── datasheets/  
│   ├── requirements/  
│   ├── standards/  
│   └── simulation/  
├── work/  
│   ├── extracted/  
│   ├── normalized/  
│   └── wcca_input.yaml  
└── output/
```

```

|   |—— calculation/
|   |—— report/
|   |—— traceability/
|   |—— risk/
|—— review/
|   |—— comments/
|   |—— approvals/
|—— release/
    |—— archive_package.zip

```

7.2 参数结构化模块

WCCA 自动化的关键是将资料转换为结构化输入。

推荐采用 YAML、JSON 或 Excel 表格描述每个电路块、器件、参数、边界条件和需求。

示例结构：

```

project:
  name: "Motor Controller"
  document_type: "WCCA"

circuit_block:
  id: "PWR_3V3"
  name: "3.3V Power Rail"
  type: "ldo_power_rail"

requirements:
  - id: "REQ-PWR-001"
    description: "3.3V output shall remain within  $\pm 5\%$ "
    min_value: 3.135
    max_value: 3.465
    unit: "V"

conditions:
  vin_min: 4.75
  vin_max: 5.25
  temperature_min: -40
  temperature_max: 85
  load_current_max: 0.8

components:
  - ref: "U12"
    part_number: "LDO_Example"
    role: "regulator"
    parameters:
      output_accuracy:
        value: 1

```

```
unit: "%"
type: "max"
source: "datasheet"
review_status: "confirmed"

- ref: "R101"
role: "feedback_top"
nominal_value: 100000
unit: "ohm"
tolerance: 1
tempco: 100
aging: 0.5
review_status: "confirmed"
```

参数结构化模块应支持：

1. 参数来源标注。
2. 参数单位统一。
3. Typical/Min/Max 类型区分。
4. 参数确认状态管理。
5. 不确定参数标记。
6. 人工复核记录。

7.3 知识库/RAG 模块

知识库用于管理 WCCA 所需的依据文档和历史经验。

知识库内容应包括：

- Datasheet
- App Note
- Reference Manual
- 公司 WCCA 模板
- 公司降额规范
- 可靠性规范
- EMC/ESD 设计规范
- 历史 WCCA 报告
- 历史设计问题复盘
- 评审意见库
- 计算规则说明
- 常用电路设计指南

知识库应支持：

1. 文档上传和版本管理。
2. 文档解析和分块。
3. 向量检索和关键词检索。
4. 混合检索和重排序。
5. 引用来源返回。

6. 参数来源追踪。
7. 历史案例复用。
8. 权限控制。

在 WCCA 流程中，知识库的作用不是直接给出最终结论，而是提供依据、上下文和引用来源。

7.4 WCCA Core CLI 模块

WCCA Core CLI 是系统的核心执行工具，建议采用 Python 实现，便于处理 Excel、文档、BOM、网表、工程计算和仿真接口。

核心功能包括：

1. ingest: 资料导入

2. 导入 BOM、网表、需求、规范、datasheet、模板。
3. 生成资料索引。
4. 检查文件完整性。

5. extract: 参数抽取

6. 从 BOM 和网表中提取器件清单。
7. 从结构化文件或知识库结果中提取参数。
8. 生成初始参数表。

9. normalize: 数据归一化

10. 统一单位。
11. 统一命名。
12. 统一字段格式。
13. 检查缺失参数。

14. calculate: 工程计算

15. 执行 Worst Case 计算。
16. 执行 RSS 计算。
17. 计算 Margin。
18. 生成 Pass/Fail 结果。

19. simulate: 仿真调用

20. 调用 SPICE 或其他仿真工具。

- 21. 执行参数扫描。
- 22. 读取仿真输出。
- 23. 生成仿真结果摘要。

24. **validate**: 自动校验

- 25. 检查数值一致性。
- 26. 检查单位一致性。
- 27. 检查引用完整性。
- 28. 检查需求覆盖率。
- 29. 检查 Pass/Fail 与 Margin 是否一致。
- 30. 检查 TBD、N/A、Typical 等风险词。

31. **report**: 报告生成

- 32. 使用 Word、Markdown 或 LaTeX 模板生成报告。
- 33. 填充计算表、结论、风险项和引用来源。
- 34. 输出 WCCA 报告草稿。

35. **package**: 归档打包

- 36. 打包正式报告、计算表、引用清单、需求覆盖矩阵、仿真结果、风险项清单和执行日志。

7.5 计算模型库

计算模型库是 WCCA 自动化复用的关键资产。

初期建议优先覆盖以下电路类型：

电路类型	自动化优先级	说明
电阻分压	高	参数明确，最适合自动化
LDO 输出电压	高	电源轨 WCCA 高频场景
DC/DC 反馈网络	高	涉及反馈电阻、参考电压、纹波和裕量
ADC 采样链路	高	包含分压、偏置、误差、量化误差
比较器阈值	高	阈值上下限和迟滞分析明确
RC 延时	高	可自动计算时间常数最坏值
运放增益电路	中	需考虑输入失调、偏置电流、开环增益等
MOSFET 开关	中	涉及损耗、热和驱动条件
热设计	中	可结合热阻和功耗计算

电路类型	自动化优先级	说明
保护电路	中	需要结合浪涌、ESD、钳位器件特性
高频/RF 电路	低	自动化难度高，更依赖仿真和专家判断

每个计算模型建议包含：

1. 模型名称。
2. 适用范围。
3. 输入参数。
4. 输出参数。
5. 计算公式。
6. 边界条件。
7. 默认单位。
8. 参数来源要求。
9. Pass/Fail 判断规则。
10. 示例输入和示例输出。
11. 单元测试用例。

7.6 AI Agent 编排模块

AI Agent 的定位是流程编排和文档表达，而不是工程计算的唯一来源。

AI Agent 可以负责：

1. 读取项目资料目录。
2. 判断需要分析的电路块。
3. 调用知识库检索参数来源。
4. 辅助生成结构化输入。
5. 调用 CLI 完成计算和校验。
6. 根据模板生成报告草稿。
7. 总结风险项。
8. 生成 Review 问题清单。
9. 检查报告是否存在明显不一致。
10. 生成面向评审会议的摘要。

AI Agent 不应负责：

1. 无依据生成关键参数。
2. 直接给出未经计算验证的 Pass/Fail。
3. 替代工程师确认边界条件。
4. 替代正式签核。
5. 自动接受风险项。
6. 自动修改设计并直接发布。

7.7 报告生成模块

WCCA 报告应采用固定模板，避免每次由 AI 自由发挥。

推荐报告章节如下：

1. 文档信息
2. 项目名称
3. 文档编号
4. 版本号
5. 作者
6. 审核人
7. 日期
8. 分析范围
9. 被分析电路模块
10. 适用产品
11. 不适用范围
12. 分析假设
13. 设计需求
14. Requirement ID
15. 指标描述
16. 最小值/最大值
17. 单位
18. 验收标准
19. 分析方法
20. Worst Case
21. RSS
22. Monte Carlo
23. SPICE 仿真
24. 降额分析
25. 参数来源
26. Datasheet 来源
27. 公司规范来源
28. 设计输入来源
29. 工程师确认记录

- 30. 电路分析
- 31. 电路功能说明
- 32. 输入条件
- 33. 输出指标
- 34. 元件参数
- 35. 计算过程
- 36. 仿真结果
- 37. 裕量分析
- 38. 计算结果
- 39. Nominal
- 40. Worst Min
- 41. Worst Max
- 42. Requirement
- 43. Margin
- 44. Pass/Fail
- 45. 风险项
- 46. 低裕量项
- 47. 参数缺失项
- 48. 使用 Typical 值项
- 49. 仿真假设风险
- 50. 需人工确认项
- 51. 结论
- 52. 是否满足设计要求
- 53. 遗留风险
- 54. 整改建议
- 55. 签核意见
- 56. 附录
- 57. 计算表
- 58. 仿真结果
- 59. 引用来源
- 60. 需求覆盖矩阵
- 61. 版本记录
- 62. 审批记录

8. 端到端业务流程

推荐端到端流程如下：

1. **收集项目资料**
工程师上传原理图、网表、BOM、datasheet、需求规格、公司规范、历史报告和仿真结果。
2. **完整性检查**
系统检查是否缺少必要输入，例如 BOM、需求规格、关键 datasheet 或 WCCA 模板。
3. **BOM/网表解析**
CLI 解析 BOM 和网表，识别器件、网络名称、电路模块和关键连接关系。
4. **电路块识别**
AI Agent 辅助识别电源轨、分压网络、ADC、比较器、运放、RC 延时等电路块。
5. **匹配计算模型**
系统根据电路类型匹配已有计算模型。若无模型，则标记为需要人工建立或手动分析。
6. **参数抽取与来源检索**
AI Agent 调用知识库检索 datasheet 和规范，辅助抽取 Min/Max/Tolerance/Tempco/Aging 等参数。
7. **工程师确认关键参数**
对关键参数、Typical 值、不确定值和缺失值进行人工确认。
8. **生成结构化 WCCA 输入**
系统生成 YAML、JSON 或 Excel 格式的结构化分析输入。
9. **执行计算和仿真**
CLI 调用 Python、Excel 或 SPICE 工具执行 Worst Case、RSS、Monte Carlo 和参数扫描。
10. **生成计算结果表**
输出 Nominal、Worst Min、Worst Max、Margin、Pass/Fail 等结果。
11. **AI 生成 WCCA 草稿**
AI Agent 根据模板、计算结果、引用来源和风险项生成 WCCA 报告草稿。
12. **自动一致性检查**
CLI 检查报告正文、计算表、引用清单和需求覆盖矩阵是否一致。
13. **工程师 Review**
工程师审核边界条件、参数来源、计算模型、风险项和最终结论。
14. **问题修正**
若存在问题，返回修改参数、模型、设计或报告内容。
15. **最终签核与归档**
生成正式 WCCA 报告、计算表、引用来源清单、审批记录和归档包。

9. 输出成果

系统应输出以下成果：

输出物	格式	说明
正式 WCCA 报告	Word/PDF	面向评审、客户或内部归档
计算工作表	Excel	包含所有关键计算过程和结果
参数来源清单	Excel/CSV	记录参数来源、页码、表格、确认状态
需求覆盖矩阵	Excel/CSV	追踪每条需求对应的分析章节和结果
风险项清单	Excel/Markdown	汇总低裕量、参数缺失、待确认项
仿真结果摘要	PDF/图片/CSV	保存关键仿真波形和参数扫描结果
审批记录	PDF/JSON	记录评审人、意见、结论和签核状态
归档包	ZIP	包含报告、计算表、引用清单、日志和版本信息

10. 数据标准与字段规范

10.1 参数字段建议

每个参数至少应包含：

- 参数名称
- 参数符号
- 参数值
- 单位
- 参数类型：Nominal、Min、Max、Tolerance、Typical
- 温度条件
- 电压条件
- 负载条件
- 来源文档
- 来源章节/页码/表格
- 提取方式：人工/AI/规则/数据库
- 确认状态：待确认/已确认/有风险/不适用
- 确认人
- 确认时间
- 备注

10.2 计算结果字段建议

每项计算结果至少应包含：

- 电路块 ID
- Requirement ID
- 分析项目

- Nominal
- Worst Min
- Worst Max
- Requirement Min
- Requirement Max
- Margin
- Pass/Fail
- 计算模型版本
- 输入参数版本
- 计算时间
- 计算工具版本
- 审核状态

11. 质量控制与自动校验

必须建立自动校验机制，避免生成看似完整但不可靠的 WCCA 报告。

关键检查项包括：

检查项	说明	是否必须
输入资料完整性	检查 BOM、网表、需求、datasheet 是否齐全	必须
参数来源完整性	检查关键参数是否有来源或确认记录	必须
Typical 值检查	检查是否将 Typical 用作 Worst Case	必须
单位一致性	检查 V/mV、A/mA、 $\Omega/k\Omega$ 等单位换算	必须
数值一致性	检查报告正文数值是否来自计算表	必须
Pass/Fail 一致性	检查结论是否与 Margin 结果一致	必须
需求覆盖率	检查每条需求是否有对应分析	必须
引用完整性	检查引用文档、页码、章节是否完整	推荐
版本一致性	检查 datasheet、BOM、模板是否为正确版本	推荐
风险词检查	检查 TBD、N/A、To Be Confirmed 等词	推荐
审批状态检查	检查是否完成工程师签核	必须

12. 实施路线规划

Phase 1：标准化阶段

目标：建立 WCCA 自动化的基础规范。

主要任务：

1. 梳理公司现有 WCCA 模板。
2. 统一报告章节结构。
3. 统一参数字段规范。
4. 统一计算表格式。
5. 建立风险项和 Review checklist。
6. 选择 2~3 个历史项目作为样例。

阶段成果：

- 标准 WCCA 报告模板
 - 参数表模板
 - 计算表模板
 - 风险项模板
 - Review checklist
 - 示例项目数据包
-

Phase 2：CLI 核心化阶段

目标：建立可复现的 WCCA 核心执行工具。

主要任务：

1. 开发 wcca init、ingest、calculate、validate、report 命令。
2. 优先实现电阻分压、电源轨、ADC、比较器等高频计算模型。
3. 自动生成计算表。
4. 自动生成初版 WCCA 报告。
5. 实现数值一致性和单位检查。
6. 建立基础单元测试和回归测试。

阶段成果：

- WCCA Core CLI
 - 初版计算模型库
 - 自动计算表
 - 自动报告草稿
 - 校验日志
 - 可复现执行流程
-

Phase 3：知识库接入阶段

目标：实现参数来源、规范依据和历史案例的可检索、可追溯。

主要任务：

1. 建设 datasheet 知识库。
2. 导入公司规范和 WCCA 模板。
3. 导入历史 WCCA 报告。

4. 建立参数来源引用规则。
5. 支持 AI Agent 调用知识库检索依据。
6. 输出参数来源清单。

阶段成果：

- Datasheet 知识库
 - 公司规范知识库
 - 历史 WCCA 案例库
 - 参数来源清单
 - 引用追溯机制
-

Phase 4: AI 编排与 Subagent 阶段

目标：实现更智能的资料理解、任务拆解、参数提取和报告生成。

主要任务：

1. 引入 WCCA Planner Agent。
2. 引入 Parameter Extraction Agent。
3. 引入 Calculation Review Agent。
4. 引入 Report Writer Agent。
5. 支持自动识别电路块。
6. 支持自动生成 Review 问题清单。
7. 支持风险项自动总结。

阶段成果：

- WCCA 多 Agent 协作流程
 - 自动电路块识别
 - 自动参数提取辅助
 - 自动风险总结
 - 自动 Review 清单
 - 更完整的报告草稿
-

Phase 5: 平台化阶段

目标：形成团队级或企业级 WCCA 自动化平台。

主要任务：

1. 建设 Web UI。
2. 接入 Git、PLM、ERP、EDA 或文档管理系统。
3. 建立权限控制。
4. 建立审批流。
5. 建立审计日志。
6. 建立项目归档和历史复用机制。
7. 建立模型版本管理。
8. 建立指标看板。

阶段成果：

- WCCA 自动化平台
- 项目管理界面
- 审批和签核流程
- 版本与权限管理
- 历史项目复用机制
- 团队级知识资产沉淀

13. 团队角色分工

角色	职责
硬件工程师	定义分析范围、确认参数、审核计算模型、签核结论
WCCA 工程师	维护模板、计算模型、分析规范和评审标准
系统工程师	提供需求规格、系统边界和验收标准
质量/可靠性工程师	审核降额、可靠性、风险项和合规要求
软件/自动化工程师	开发 CLI、计算引擎、报告生成和集成工具
AI 工程师	建设 Agent、知识库检索、提示词和流程编排
文档管理员	管理模板、版本、归档和发布流程
项目负责人	确认流程推广、资源协调和最终交付

14. 验收标准

系统初版可按以下标准验收：

1. 能够导入 BOM、需求、参数表和 WCCA 模板。
2. 能够对至少 3 类常见电路完成自动计算。
3. 能够生成标准 WCCA 报告草稿。
4. 报告中的关键数值能追溯到计算表。
5. 参数能追溯到来源文档或人工确认记录。
6. 能够输出计算表、风险项清单和需求覆盖矩阵。
7. 能够检查单位错误、数值不一致和 Typical 值风险。
8. 同一输入多次运行结果一致。
9. 工程师可在报告中完成 Review 和签核。
10. 至少通过一个历史项目回归验证。

正式推广阶段可增加以下验收标准：

1. 支持多项目复用。
2. 支持历史案例检索。
3. 支持 datasheet 和规范知识库引用。
4. 支持审批流。

- 5. 支持版本管理。
 - 6. 支持与 Git、PLM 或文档系统集成。
 - 7. 支持审计日志导出。
-

15. 风险与控制措施

15.1 AI 生成错误参数

风险：AI 可能误读 datasheet 或生成不存在的参数。

控制措施：

- 所有关键参数必须有来源。
- 参数必须经过工程师确认。
- Typical 值必须自动标记。
- 不确定参数不得自动通过。

15.2 计算模型错误

风险：计算公式或边界条件不正确会导致错误结论。

控制措施：

- 每个计算模型必须有版本号。
- 每个模型必须有单元测试。
- 关键模型必须由工程师审核。
- 历史项目用于回归测试。

15.3 报告与计算表不一致

风险：报告正文中的数值与计算表不一致。

控制措施：

- 报告数值必须通过模板变量填充。
- 禁止 AI 自由生成关键数值。
- validate 命令自动检查一致性。

15.4 引用来源不完整

风险：参数和结论无法追溯。

控制措施：

- 参数表中强制包含来源字段。
- 引用缺失则报告不能进入正式发布状态。
- 知识库保留文档版本。

15.5 自动化过度替代人工判断

风险：工程师过度依赖 AI，忽略边界条件和特殊工况。

控制措施：

- 工程师签核为强制流程。

- 高风险项必须人工确认。
 - AI 只生成建议，不自动接受风险。
-

16. 推荐技术栈

16.1 初期技术栈

- Python
- Pandas / OpenPyXL
- python-docx
- Jinja2
- YAML / JSON
- Git
- Markdown
- Excel
- Claude Code 或 Codex
- 本地文件夹知识库

16.2 中期技术栈

- RAGFlow 或企业知识库
- 向量数据库
- PostgreSQL
- MinIO / NAS
- LTspice / ngspice / 其他仿真工具
- PySpice 或仿真调用脚本
- MCP 工具接口
- CI/CD 流程

16.3 企业级技术栈

- Web UI
 - 权限系统
 - 审批流
 - PLM/ERP/MES/EDA 集成
 - 文档管理系统
 - 审计日志
 - 模型版本管理
 - 项目看板
 - API 服务化架构
-

17. 推荐落地顺序

建议不要一开始就建设复杂平台，而应按以下顺序逐步落地：

1. 先标准化模板和参数表

明确 WCCA 报告格式、参数字段、计算表结构和审核要求。

- 2. 再开发 CLI 核心工具
优先实现可复现计算、校验和报告生成。
- 3. 再接入知识库/RAG
实现 datasheet、规范和历史案例的可检索、可追溯。
- 4. 再引入 AI Agent 和 Subagent
用 AI 提高资料理解、参数提取、报告生成和审查效率。
- 5. 最后平台化
增加 Web UI、审批流、权限控制、归档发布和系统集成。

18. 最终推荐方案

针对硬件 WCCA 文档生成，推荐采用以下方案：

以 WCCA Core CLI 为核心执行引擎，以知识库/RAG 提供依据支撑，以 AI Agent 负责编排和文档表达，以 Skill 固化流程规范，以 Subagent 承担复杂任务分工，以工程师签核保证最终责任闭环。

最终系统形态建议为：

WCCA Automation Toolkit

—— wcca-cli	# 核心执行工具
—— wcca-skill	# 流程规范、模板规则、检查清单
—— wcca-agents	# Planner / Extractor / Reviewer / Writer
—— wcca-models	# 电路计算模型库
—— wcca-templates	# WCCA 报告模板库
—— wcca-knowledge-base	# Datasheet / 公司规范 / 历史案例
—— wcca-validation	# 单位、数值、引用、需求覆盖检查
—— wcca-report-generator	# Word / PDF / Markdown 报告生成
—— wcca-review-workflow	# 人工评审、审批、签核、归档

一句话总结：

WCCA 文档生成不应做成单纯的 AI 写作工具，而应建设为“知识库给依据、CLI 做计算与校验、AI 负责编排和表达、工程师最终签核”的可追溯工程分析系统。